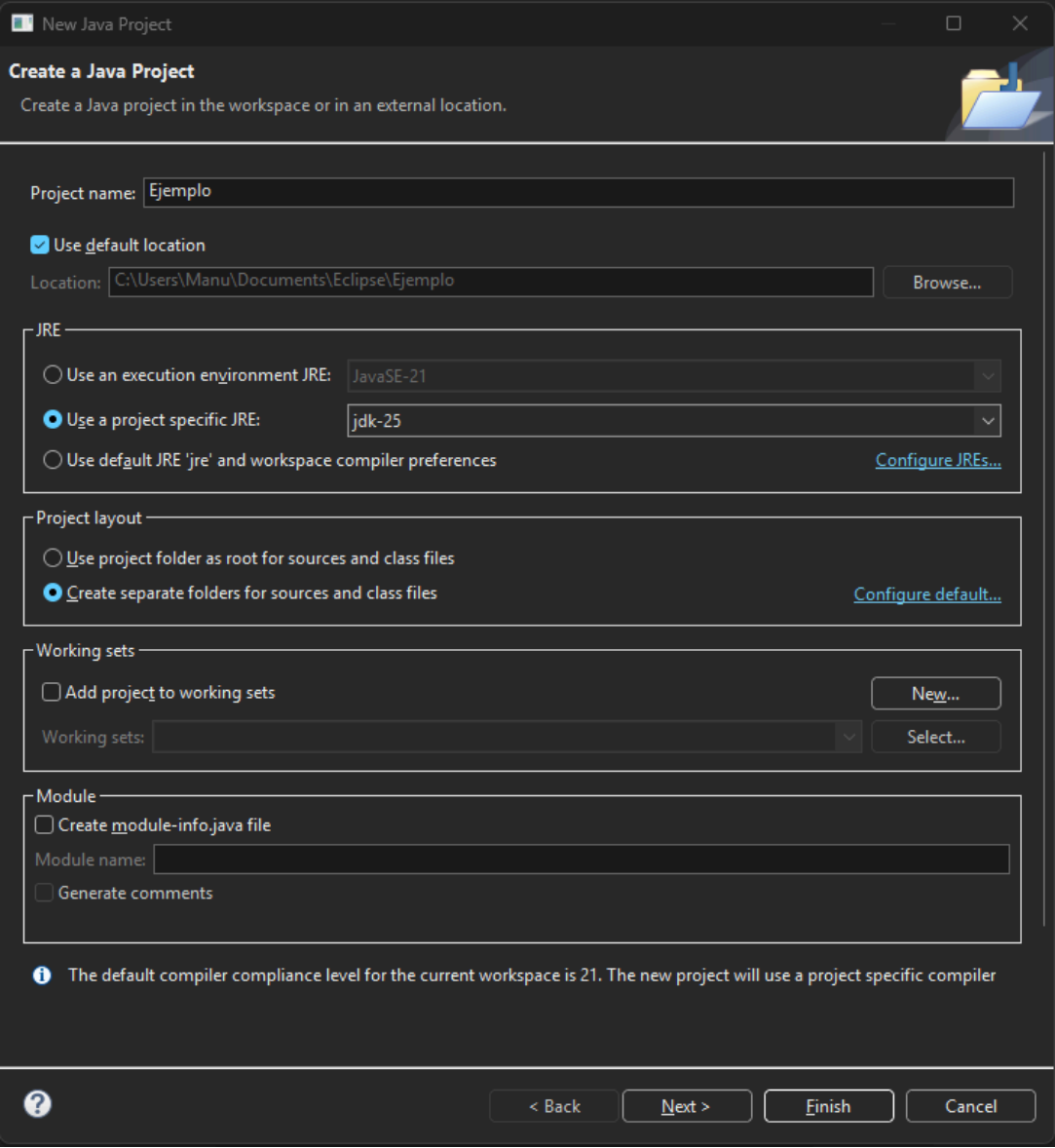


ENTORNOS DE DESARROLLO

1. TEST UNITARIOS CON JUNIT

Creación del proyecto



The screenshot shows the 'New Java Project' dialog box in Eclipse. The title bar says 'New Java Project'. The main heading is 'Create a Java Project' with a subtitle 'Create a Java project in the workspace or in an external location.' and a folder icon. The 'Project name' field contains 'Ejemplo'. The 'Use default location' checkbox is checked, and the 'Location' field shows 'C:\Users\Manu\Documents\Eclipse\Ejemplo' with a 'Browse...' button. The 'JRE' section has three radio buttons: 'Use an execution environment JRE:' (selected 'JavaSE-21'), 'Use a project specific JRE:' (selected 'jdk-25'), and 'Use default JRE 'jre' and workspace compiler preferences' (with a 'Configure JREs...' link). The 'Project layout' section has two radio buttons: 'Use project folder as root for sources and class files' and 'Create separate folders for sources and class files' (selected, with a 'Configure default...' link). The 'Working sets' section has an unchecked 'Add project to working sets' checkbox, a 'Working sets:' dropdown, and 'New...' and 'Select...' buttons. The 'Module' section has an unchecked 'Create module-info.java file' checkbox, a 'Module name:' field, and an unchecked 'Generate comments' checkbox. A message at the bottom states: 'The default compiler compliance level for the current workspace is 21. The new project will use a project specific compiler'. At the bottom are buttons for '?', '< Back', 'Next >', 'Finish', and 'Cancel'.

New Java Project

Create a Java Project

Create a Java project in the workspace or in an external location.

Project name:

☒ Use default location

Location: [Browse...](#)

JRE

☐ Use an execution environment JRE:

☒ Use a project specific JRE:

☐ Use default JRE 'jre' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets [New...](#)

Working sets: [Select...](#)

Module

☐ Create module-info.java file

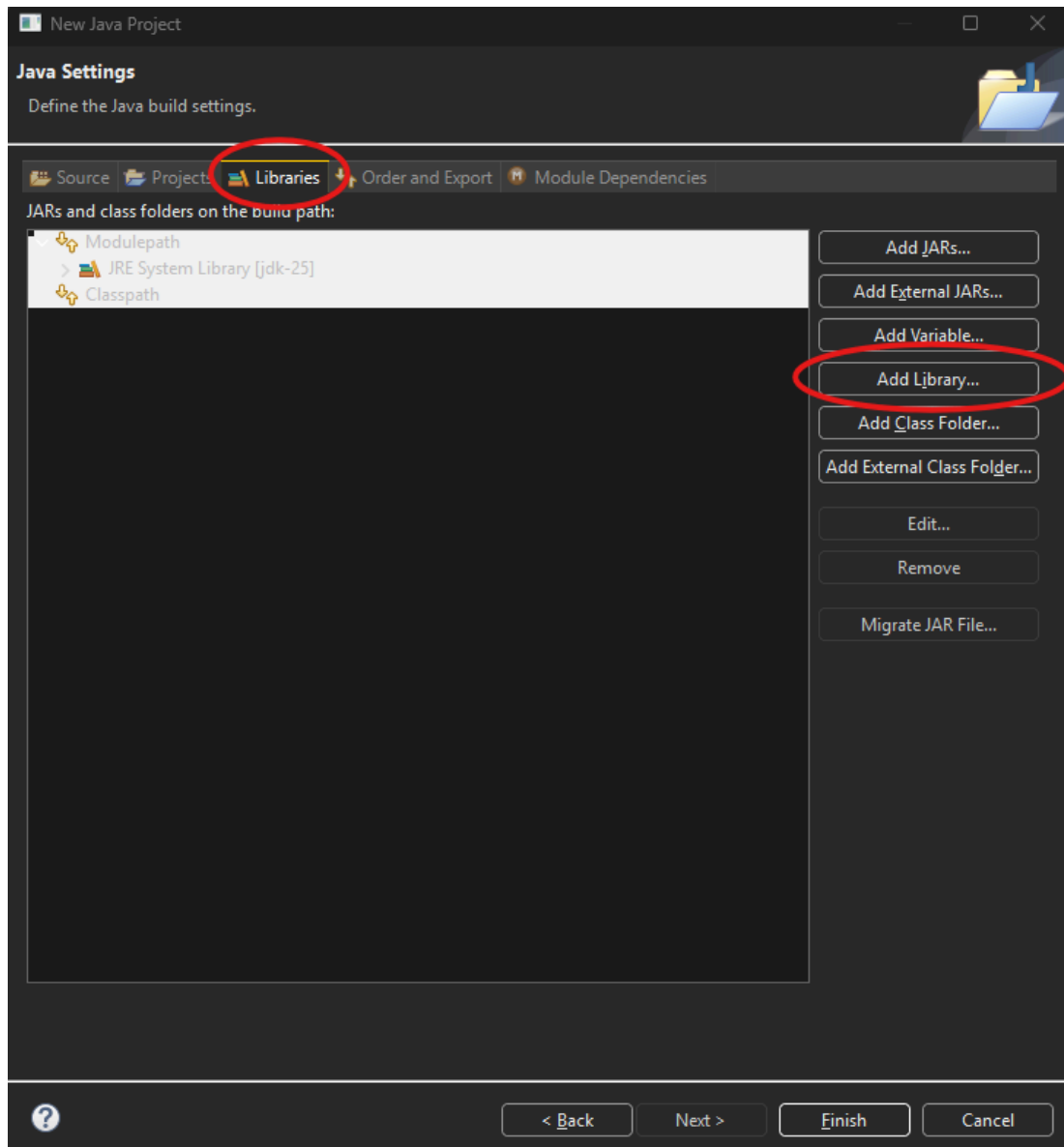
Module name:

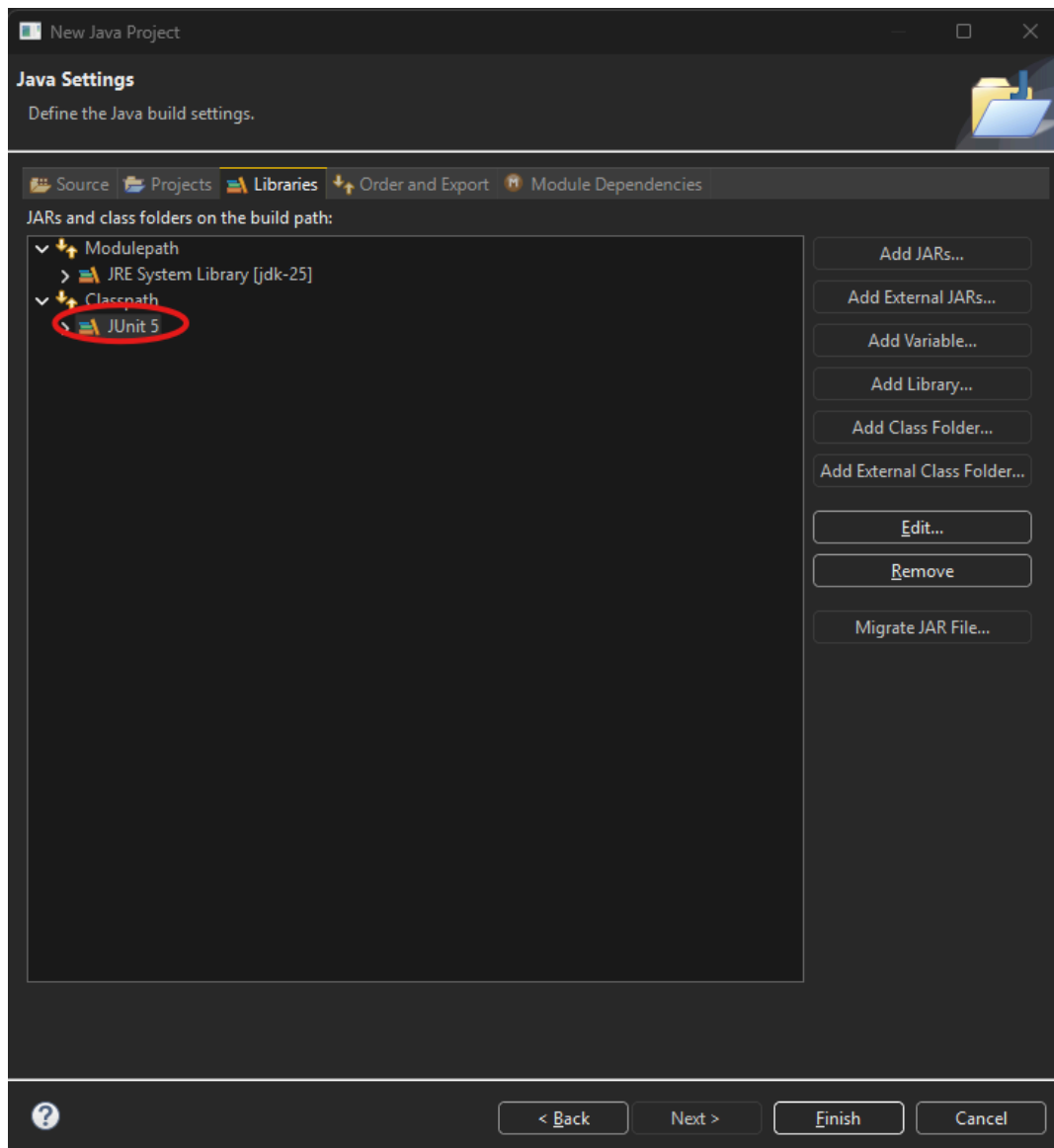
☐ Generate comments

i The default compiler compliance level for the current workspace is 21. The new project will use a project specific compiler

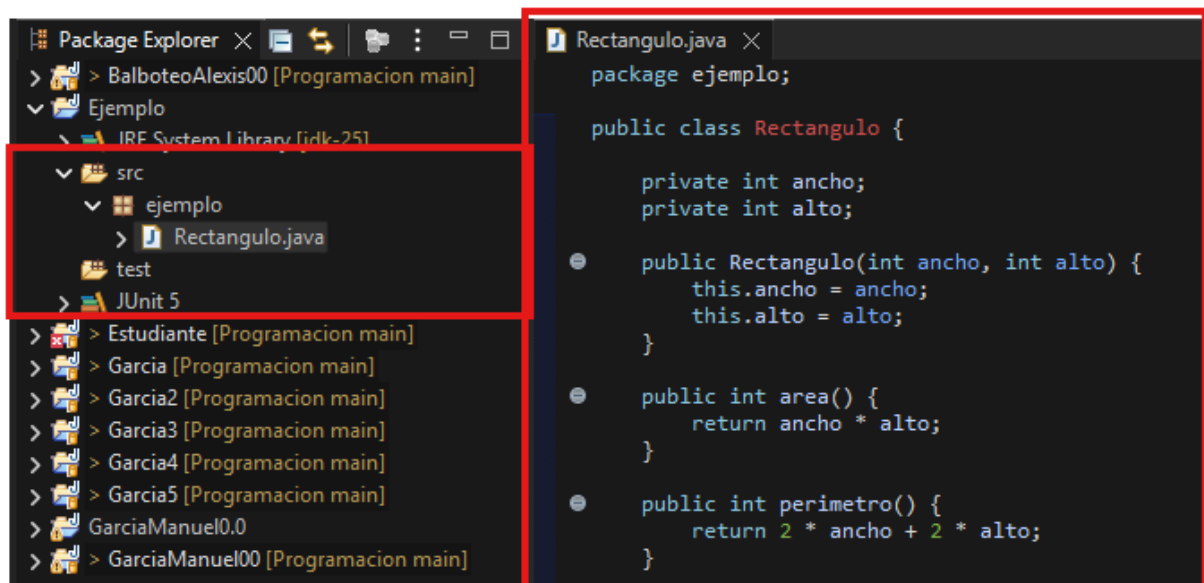
? < Back Next > Finish Cancel

Añadimos la librería de JUnit

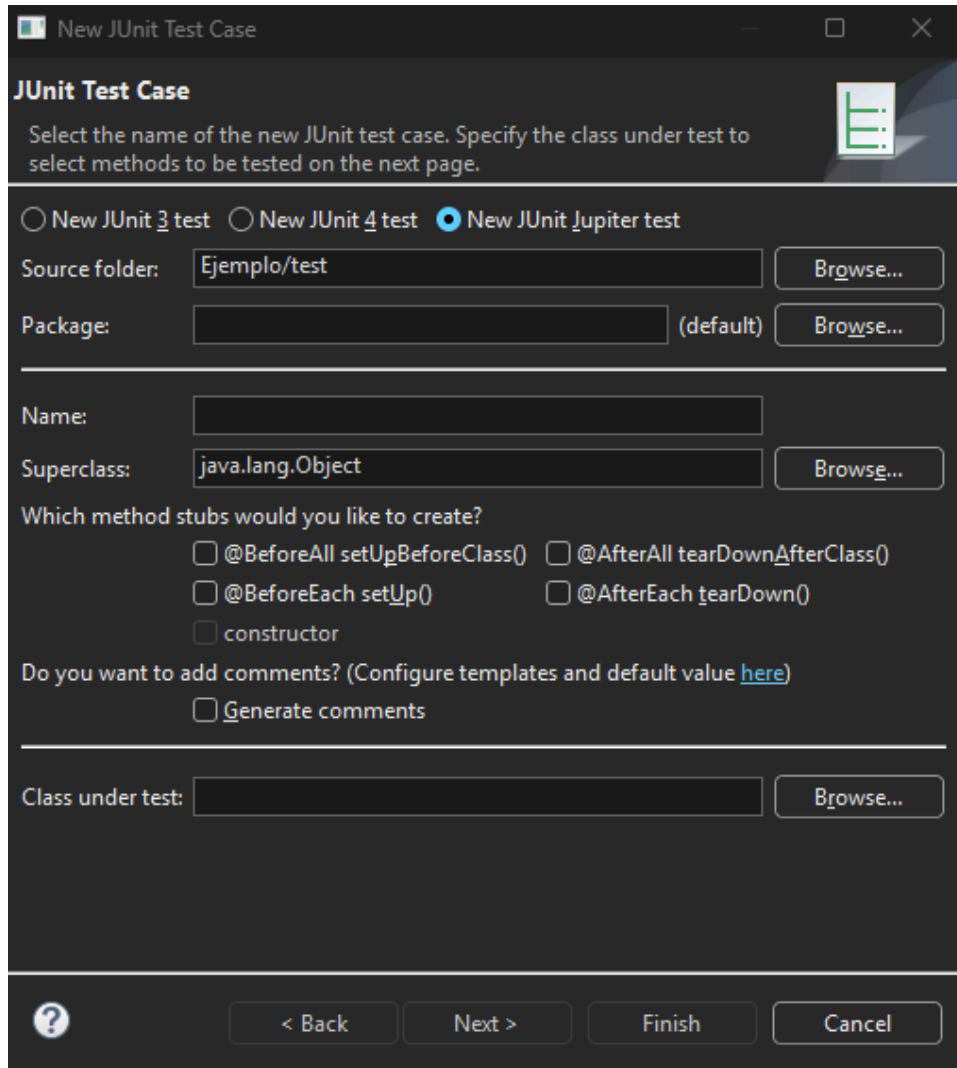




Creamos la clase Rectangulo



Creamos el paquete de test



New JUnit Test Case

Select the name of the new JUnit test case. Specify the class under test to select methods to be tested on the next page.

☐ New JUnit 3 test ☐ New JUnit 4 test ☒ New JUnit Jupiter test

Source folder:

Package:

Name:

Superclass:

Which method stubs would you like to create?

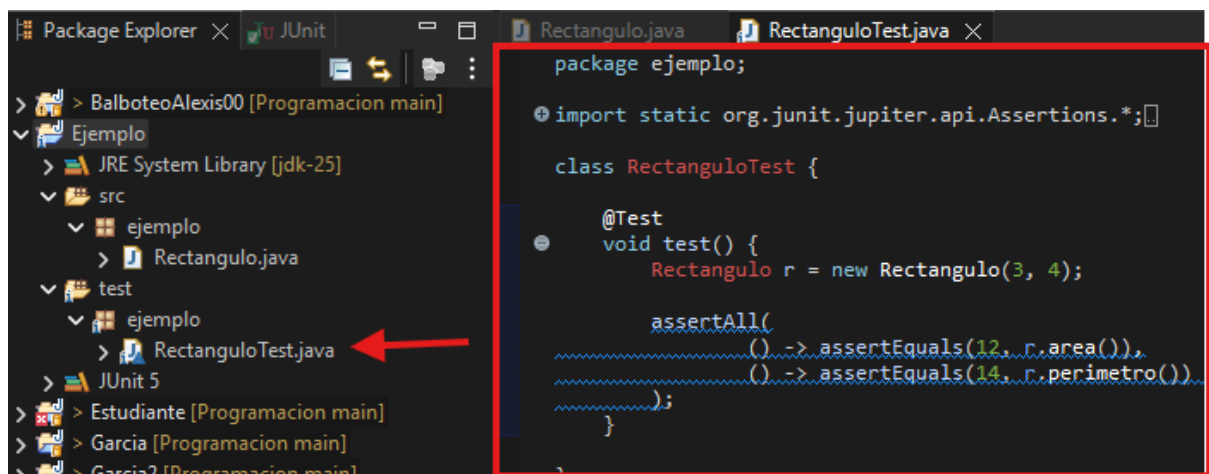
☐ @BeforeAll setUpBeforeClass() ☐ @AfterAll tearDownAfterClass()
☐ @BeforeEach setUp() ☐ @AfterEach tearDown()
☐ constructor

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

Class under test:

Creamos la clase RectanguloTest



```
package ejemplo;

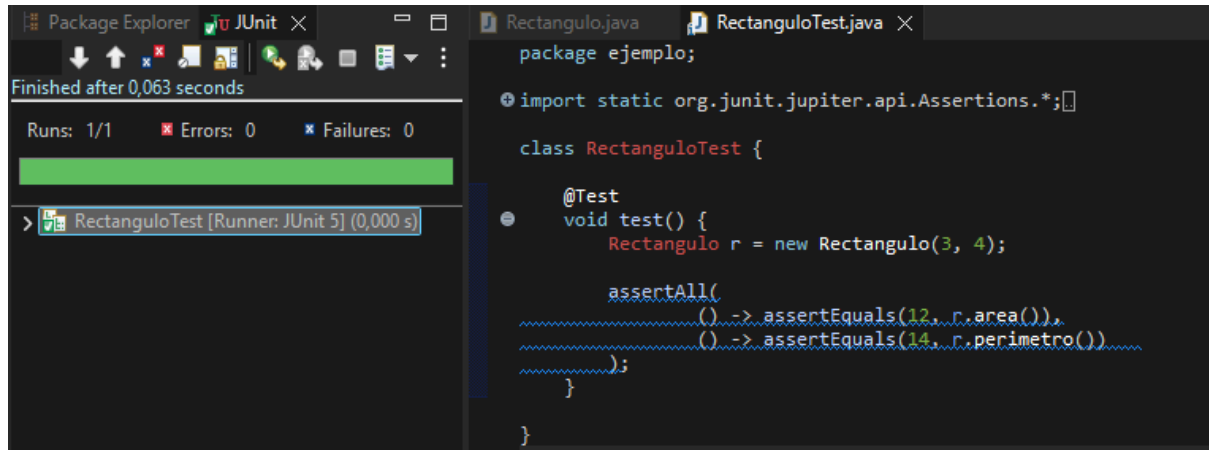
import static org.junit.jupiter.api.Assertions.*;

class RectanguloTest {

    @Test
    void test() {
        Rectangulo r = new Rectangulo(3, 4);

        assertEquals(12, r.area());
        assertEquals(14, r.perimetro());
    }
}
```

Comprobamos con JUnit



```
package ejemplo;

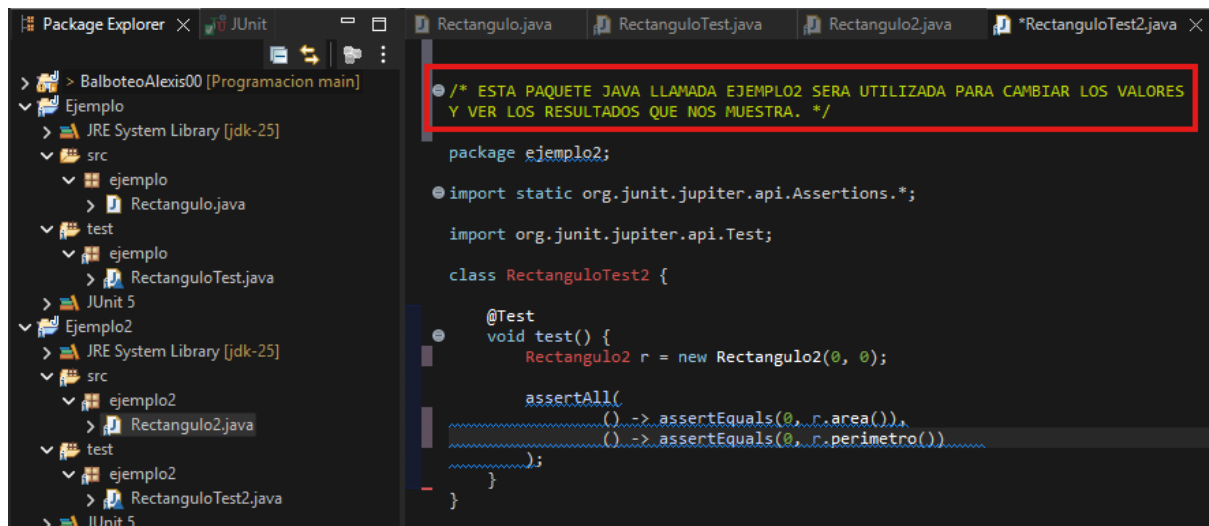
import static org.junit.jupiter.api.Assertions.*;

class RectanguloTest {

    @Test
    void test() {
        Rectangulo r = new Rectangulo(3, 4);

        assertEquals(12, r.area());
        assertEquals(14, r.perimetro());
    }
}
```

Creamos otro proyecto ejemplo2 para realizar test con datos erróneos.



```
/* ESTA PAQUETE JAVA LLAMADA EJEMPLO2 SERA UTILIZADA PARA CAMBIAR LOS VALORES
Y VER LOS RESULTADOS QUE NOS MUESTRA. */

package ejemplo2;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

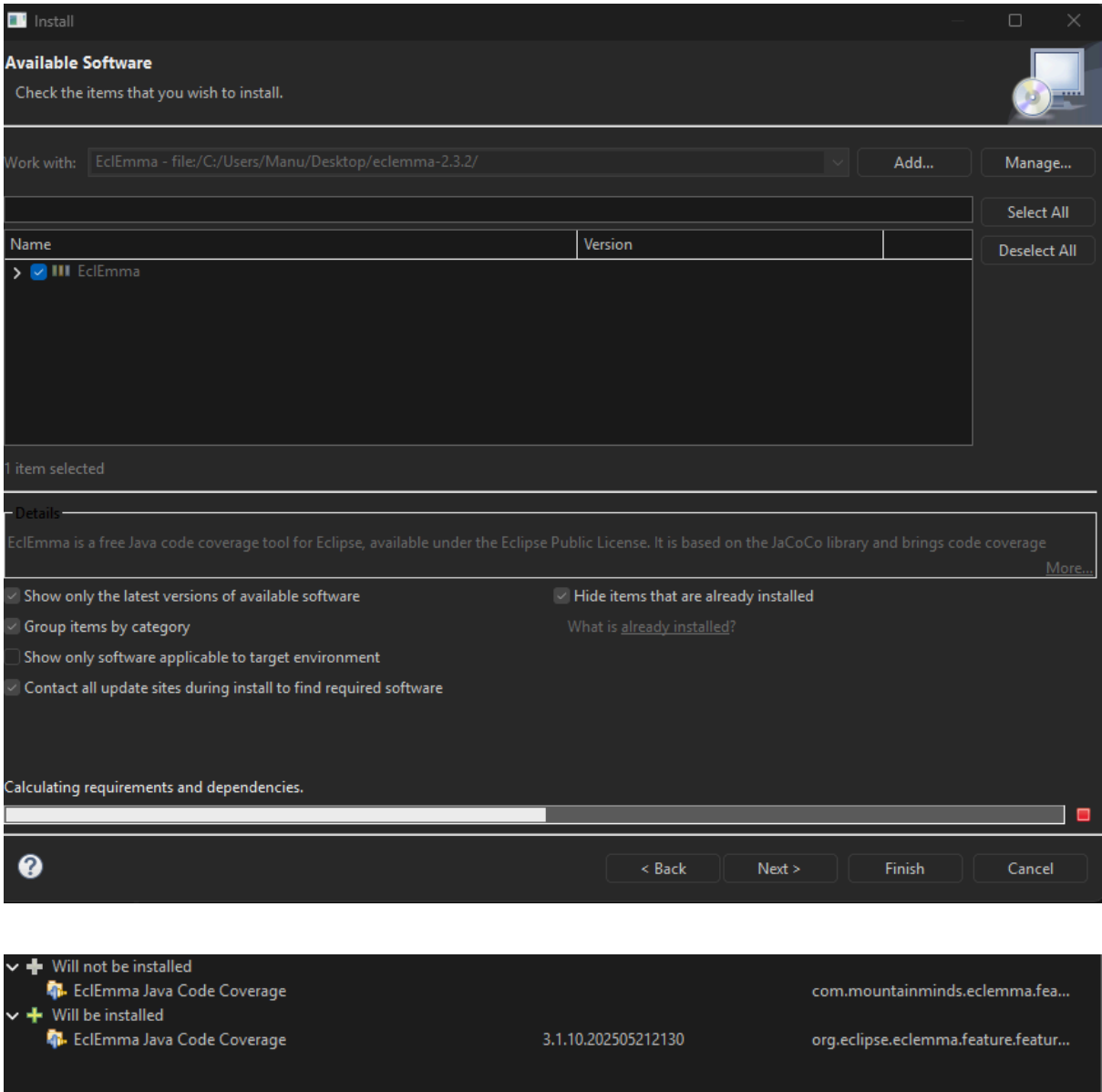
class RectanguloTest2 {

    @Test
    void test() {
        Rectangulo2 r = new Rectangulo2(0, 0);

        assertEquals(0, r.area());
        assertEquals(0, r.perimetro());
    }
}
```

2. EcEmma PARA COBERTURAS DE TEST

Instalamos EcEmma



Hacemos coverage sobre ejemplo y ejemplo2 para ver ambos resultados

The screenshot shows the Eclipse IDE interface. The main editor displays the source code of `RectanguloTest2.java`. The code includes a package declaration, imports, and a test method that uses `assertEquals` to verify the area and perimeter of a `Rectangulo2` object.

```
1
2
3 /* ESTA PAQUETE JAVA LLAMADA EJEMPLO2 SERA UTILIZADA PARA CAMBIAR LOS VALORES
4 Y VER LOS RESULTADOS QUE NOS MUESTRA. */
5
6 package ejemplo2;
7
8 import static org.junit.jupiter.api.Assertions.*;
9
11
12 class RectanguloTest2 {
13
14     @Test
15     void test() {
16         Rectangulo2 r = new Rectangulo2(5, 0);
17
18         assertEquals(r.area(), 25);
19         assertEquals(r.perimetro(), 10);
20     }
21 }
22
23
```

The left sidebar shows the Package Explorer with the project structure. The bottom panel displays the Coverage report for the test run.

Coverage Report:

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Ejemplo	100,0 %	58	0	58

Eclipse - Ejemplo2/test/ejemplo2/RectanguloTest2.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer JUnit

Finished after 0,12 seconds

Runs: 1/1 Errors: 0 Failures: 1

RectanguloTest2 [Runner: JUnit 5] (0,000 s)

test() (0,000 s)

Failure Trace

org.opentest4j.MultipleFailuresError: Multiple Failures (2 failures)
org.opentest4j.AssertionFailedError: expected: <0>
at org.junit.jupiter.api.AssertAll.assertAll(AssertAll.java:45)
at ejemplo2.RectanguloTest2.test(RectanguloTest2.java:20)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1541)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1541)
Suppressed: org.opentest4j.AssertionFailedError: expected: <0>
at ejemplo2.RectanguloTest2.lambda\$1(RectanguloTest2.java:20)
at java.base/java.util.stream.ReferencePipeline\$3\$1.accept(ReferencePipeline.java:197)
at java.base/java.util.Spliterators\$ArraySpliterator.forEachRemaining(Spliterators.java:991)
at java.base/java.util.stream.AbstractPipeline.copyInto(AbstractPipeline.java:484)
at java.base/java.util.stream.AbstractPipeline.wrapAndCopyInto(AbstractPipeline.java:474)
at java.base/java.util.stream.ReduceOps\$ReduceOp.evaluateSequential(ReduceOps.java:913)
at java.base/java.util.stream.AbstractPipeline.evaluate(AbstractPipeline.java:234)
at java.base/java.util.stream.ReferencePipeline\$1.evaluate(ReferencePipeline.java:56)
... 7 more

```
1
2
3 0 /* ESTA PAQUETE JAVA LLAMADA EJEMPLO2 SERA UTILIZADA PARA CAMBIAR LOS VALORES
4 Y VER LOS RESULTADOS QUE NOS MUESTRA. */
5
6 package ejemplo2;
7
8 0 import static org.junit.jupiter.api.Assertions.*;
9
10
11
12 class RectanguloTest2 {
13
14     @Test
15     void test() {
16         Rectangulo2 r = new Rectangulo2(5, 0);
17
18         assertAll(
19             () -> assertEquals(0, r.area()),
20             () -> assertEquals(0, r.perimetro())
21         );
22     }
23 }
```

Problems Javadoc Declaration Console SonarQube Report SonarQube Rule Description Coverage

Ejemplo2 (4 jun 2026 13:51:24)

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
> Ejemplo2	87,9 %	51	7	58

3. SONARLINT EN ACCIÓN

SONARLINT: Es un corrector mientras escribes código. Se centra en corregir el archivo en el que estás trabajando.

Ejemplos de notificaciones mientras programas.

- "Aquí tienes una variable que nunca usas"
- "Este método es demasiado complejo"
- "Este código no sigue las reglas de calidad definidas"

SONARQUBE: Es el auditor de todo el proyecto. No mira solamente un archivo individual, si no que hace un análisis de todo el proyecto. En cambio Sonarqube cubre más ámbitos.

- Todo el proyecto
- Todo el historial de análisis
- Cobertura de test
- Vulnerabilidades de seguridad
- Duplicación de código
- etc.

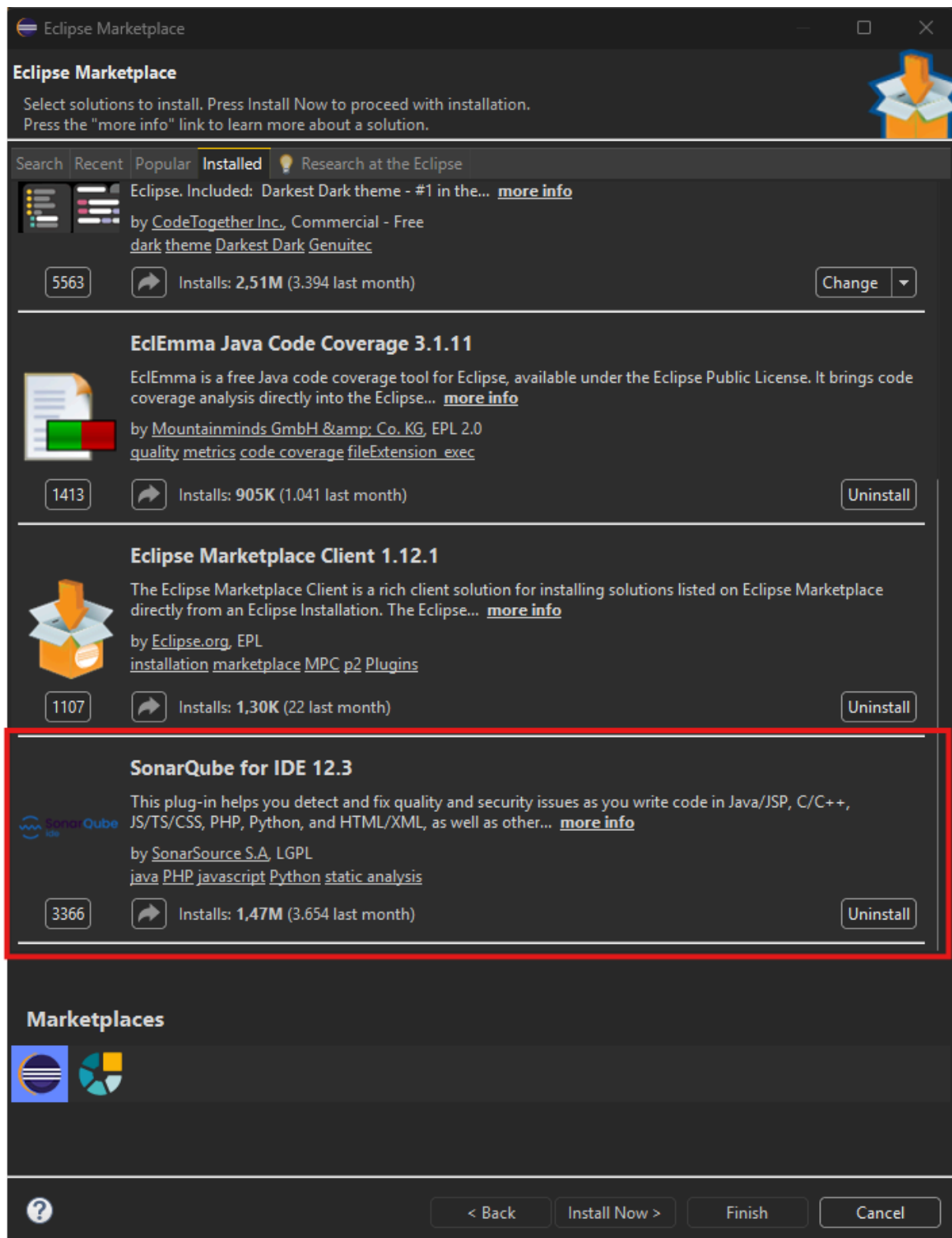
Cuando usar Sonarlint:

- Mientras estás desarrollando
- Antes de hacer un commit
- Para detectar problemas al momento

Cuando usar Sonarqube:

- En proyectos medianos o grandes
- Cuando varias personas trabajan sobre el mismo código
- En integración continua (CI/CD)

Instalamos desde marketplace Sonarqube (SonarLint no tenía la posibilidad de instalarlo).



Eclipse Marketplace

Select solutions to install. Press Install Now to proceed with installation.
Press the "more info" link to learn more about a solution.

Search Recent Popular **Installed** Research at the Eclipse

Eclipse. Included: Darkest Dark theme - #1 in the... [more info](#)
by [CodeTogether Inc.](#), Commercial - Free
[dark theme](#) [Darkest Dark](#) [Genuitec](#)
5563 Installs: 2,51M (3.394 last month) [Change](#)

EclEmma Java Code Coverage 3.1.11
EclEmma is a free Java code coverage tool for Eclipse, available under the Eclipse Public License. It brings code coverage analysis directly into the Eclipse... [more info](#)
by [Mountainminds GmbH & Co. KG](#), EPL 2.0
[quality metrics](#) [code coverage](#) [fileExtension](#) [exec](#)
1413 Installs: 905K (1.041 last month) [Uninstall](#)

Eclipse Marketplace Client 1.12.1
The Eclipse Marketplace Client is a rich client solution for installing solutions listed on Eclipse Marketplace directly from an Eclipse Installation. The Eclipse... [more info](#)
by [Eclipse.org](#), EPL
[installation](#) [marketplace](#) [MPC](#) [p2](#) [Plugins](#)
1107 Installs: 1,30K (22 last month) [Uninstall](#)

SonarQube for IDE 12.3
This plug-in helps you detect and fix quality and security issues as you write code in Java/JSP, C/C++, JS/TS/CSS, PHP, Python, and HTML/XML, as well as other... [more info](#)
by [SonarSource S.A.](#), LGPL
[java](#) [PHP](#) [javascript](#) [Python](#) [static analysis](#)
3366 Installs: 1,47M (3.654 last month) [Uninstall](#)

Marketplaces

[?](#) [< Back](#) [Install Now >](#) [Finish](#) [Cancel](#)

Comprobamos los reportes, y la descripción de la regla.

The screenshot shows the Eclipse IDE interface with the SonarQube Report tab active. It displays a table of 3 items with columns for Resource, Date, and Description. The items are related to package naming conventions. Below the table, the 'SonarQube Rule Description' tab is open, showing the rule 'Package names should comply with a naming convention' (java:S120). The description explains that shared naming conventions improve readability and that the rule checks package names against a regular expression. The parameters section shows the format: `^[a-z_]+\.[a-z_][a-z0-9_]*$`.

Resource	Date	Description
Rectangulo2	6 hours ago	✓ Rename this package name to match the regular expression <code>^[a-z_]+\.[a-z_][a-z0-9_]*\$</code> .
RectanguloT	6 hours ago	✓ Rename this package name to match the regular expression <code>^[a-z_]+\.[a-z_][a-z0-9_]*\$</code> .
RectanguloT	6 hours ago	✗ Remove this assertion from production code. [+2 locations]

Package names should comply with a naming convention java:S120

Consistency | Not identifiable | Maintainability

Why is this an issue? How can I fix it?

Shared naming conventions improve readability and allow teams to collaborate efficiently. This rule checks that all package names match a provided regular expression.

Parameters

Parameter values can be set in [Rules Configuration](#). In connected mode, server-side configuration overrides local settings.

format `^[a-z_]+\.[a-z_][a-z0-9_]*$`

4. REFACTORIZANDO JAVA CON ECLIPSE

Creemos el proyecto Ejemplo 3 para refactorizar todo el código que teníamos en Ejemplo.

The screenshot shows the Eclipse IDE with the Package Explorer on the left and the Editor on the right. The Package Explorer shows a project named 'Ejemplo3' with a 'src' folder containing 'Rectangulo.java' and a 'test' folder containing 'RectanguloTest.java'. The Editor shows the code for 'Rectangulo.java', which defines a 'Rectangle' class with methods for calculating surface and perimeter, and a private method for validating dimensions.

```
1 package ejemplo3;
2
3 public class Rectangulo {
4
5     private static final int TWO = 2;
6
7     private int width;
8     private int height;
9
10    public Rectangulo(int width, int height) {
11        this.width = width;
12        this.height = height;
13    }
14
15    public int surface() {
16        validateDimensions();
17        return width * height;
18    }
19
20    public int perimeter() {
21        validateDimensions();
22        return TWO * width + TWO * height;
23    }
24
25    private void validateDimensions() {
26        if (width <= 0 || height <= 0) {
27            throw new IllegalArgumentException(
28                "Width and height must be greater than zero");
29        }
30    }
31 }
```

The screenshot shows an IDE with a Package Explorer on the left and a code editor on the right. The Package Explorer shows a project named 'ejemplo3' with a 'test' folder containing 'RectangleTest.java'. The code editor shows the following Java code:

```
1 package ejemplo3;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 import org.junit.jupiter.api.Test;
6
7 public class RectangleTest {
8
9     @Test
10    void test() {
11        Rectangle r = new Rectangle(3, 4);
12
13        assertEquals(12, r.surface());
14        assertEquals(14, r.perimeter());
15    }
16 }
17
18 }
```

The screenshot shows the same IDE with the JUnit test run results. The Package Explorer shows 'RectangleTest [Runner: JUnit 5] (0,000 s)'. The code editor shows the same Java code as the previous screenshot.

Finished after 0,069 seconds

Runs: 1/1 Errors: 0 Failures: 0

The screenshot shows the same IDE with the JUnit test run results. The Package Explorer shows 'RectangleTest [Runner: JUnit 5] (0,000 s)' with a sub-entry 'test() (0,000 s)'. The code editor shows the same Java code as the previous screenshots.

Finished after 0,078 seconds

Runs: 1/1 Errors: 0 Failures: 1

5. PRÁCTICA JAVADOC Y MARKDOWN

Creamos la documentación en JAVADOC.

```
RectanguloTest. RectanguloTest2. RectangleTest.j RectanguloTest4 Rectangulo4.jav × »3
1 package ejemplo4;
2
3 /**
4  * Representa un rectángulo definido por su ancho y su alto.
5  *
6  * @author Manuel García García
7  * @version 1.0
8  */
9 public class Rectangulo4 {
10
11     private int ancho;
12     private int alto;
13
14     /**
15      * Crea un rectángulo con las dimensiones indicadas.
16      *
17      * @param ancho ancho del rectángulo
18      * @param alto alto del rectángulo
19      */
20     public Rectangulo4(int ancho, int alto) {
21         this.ancho = ancho;
22         this.alto = alto;
23     }
24
25     /**
26      * Calcula el área del rectángulo.
27      *
28      * @return área del rectángulo
29      */
30     public int area() {
31         return ancho * alto;
32     }
33
34     /**
35      * Calcula el perímetro del rectángulo.
36      *
37      * @return perímetro del rectángulo
38      */
39     public int perimetro() {
40         return 2 * ancho + 2 * alto;
41     }
42 }
43
```

```
RectanguloTest. RectanguloTest2. RectangleTest.j RectanguloTest4 × Rectangulo4.jav × »3
1 package ejemplo4;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 import org.junit.jupiter.api.Test;
6
7 /**
8  * Pruebas unitarias para la clase Rectangulo4.
9  *
10  * @author Manuel García García
11  * @version 1.0
12  */
13 public class RectanguloTest4 {
14
15     /**
16      * Comprueba que el cálculo del área y del perímetro
17      * de un rectángulo es correcto.
18      */
19     @Test
20     void test() {
21         Rectangulo4 r = new Rectangulo4(3, 4);
22
23         assertEquals(12, r.area());
24         assertEquals(14, r.perimetro());
25     }
26 }
27
28
```

